

SPECIFIKACIJA PROJEKTA

Softverski agenti — 2026
Distribuirana detekcija spam SMS poruka
putem federativnog učenja i aktorskog radnog okvira

Revizija dokumenta — ažurirano prema izrađenoj implementaciji. Izmene u odnosu na originalni predlog su označene zelenom pozadinom i oznakom (novo u ovoj reviziji).

1. Tim

Ime i prezime	Aleksandar Žebeljan
Broj indeksa	RA 155/2022

2. Zadatak

Cilj projekta je implementacija generičkog aktorskog radnog okvira u programskom jeziku Go, demonstriranog kroz sistem federativnog učenja za detekciju spam SMS poruka. Ciljna ocena je 10. Sistem podržava klasterovanje u P2P i provider modu, CRDT strukture za sinhronizaciju distribuiranog stanja, i izvršavanje na više mašina.

Element	Status
Federativno učenje (Naive Bayes + FedAvg)	Implementirano i testirano
Aktorski radni okvir — obavezni elementi	Implementirano i testirano
CRDT (G-Counter)	Implementirano i testirano
Klasterovanje — Provider mod	Implementirano i testirano preko gRPC/TCP
Klasterovanje — P2P gossip mod	Implementirano i testirano preko hashicorp/memberlist
Udaljena komunikacija (gRPC + Protobuf)	Implementirano i testirano
Izvršavanje na više mašina/procesa	Testirano: 5 realnih procesa + 5 Docker kontejnera

3. Federativno učenje

► 3.1 Problem koji se rešava

Detekcija spam SMS poruka bez centralnog čuvanja korisničkih podataka. Svaki Worker čvor trenira model isključivo na svom lokalnom skupu poruka, dok koordinatorski čvor agregira parametre modela bez ikad uvida u sirove poruke. Time se garantuje privatnost podataka uz zadržavanje preciznosti klasifikatora.

► 3.2 Algoritam — Federativni Naive Bayes

Naive Bayes je idealan za federativno okruženje jer su njegovi parametri aditivni — agregacija se svodi na sabiranje frekvencija reči po čvorovima, bez prenosa gradijenta.

Lokalni korak (WorkerActor):

- Za svaku poruku iz lokalne particije broji reči po klasi (spam / ham)
- Pošalji $\text{wordCounts}[\text{klasa}][\text{reč}] = \text{broj}$ i $\text{classTotals}[\text{klasa}]$ Agregatoru

Agregacijski korak (AggregatorActor — FedAvg):

$$P(t \mid c) = (\sum_i \text{count}_i(t, c) + 1) / (\sum_i \text{total}_i(c) + |V|)$$

gde je $|V|$ veličina globalnog rečnika, a $\text{total}_i(c)$ ukupan broj tokena klase c na čvoru i . Laplace glaćenje (+1) sprečava nulte verovatnoće za neviđene termine.

► 3.3 Skup podataka

Naziv	SMS Spam Collection Dataset
Izvor	UCI Machine Learning Repository — archive.ics.uci.edu/dataset/228
Veličina	5.574 SMS poruka — 747 spam, 4.827 ham
Format	TSV: label\ttext
Split	Train 80% (distribuiran ravnomerno po Worker čvorovima) / Test 20% (Coordinator)

► 3.4 Metod evaluacije

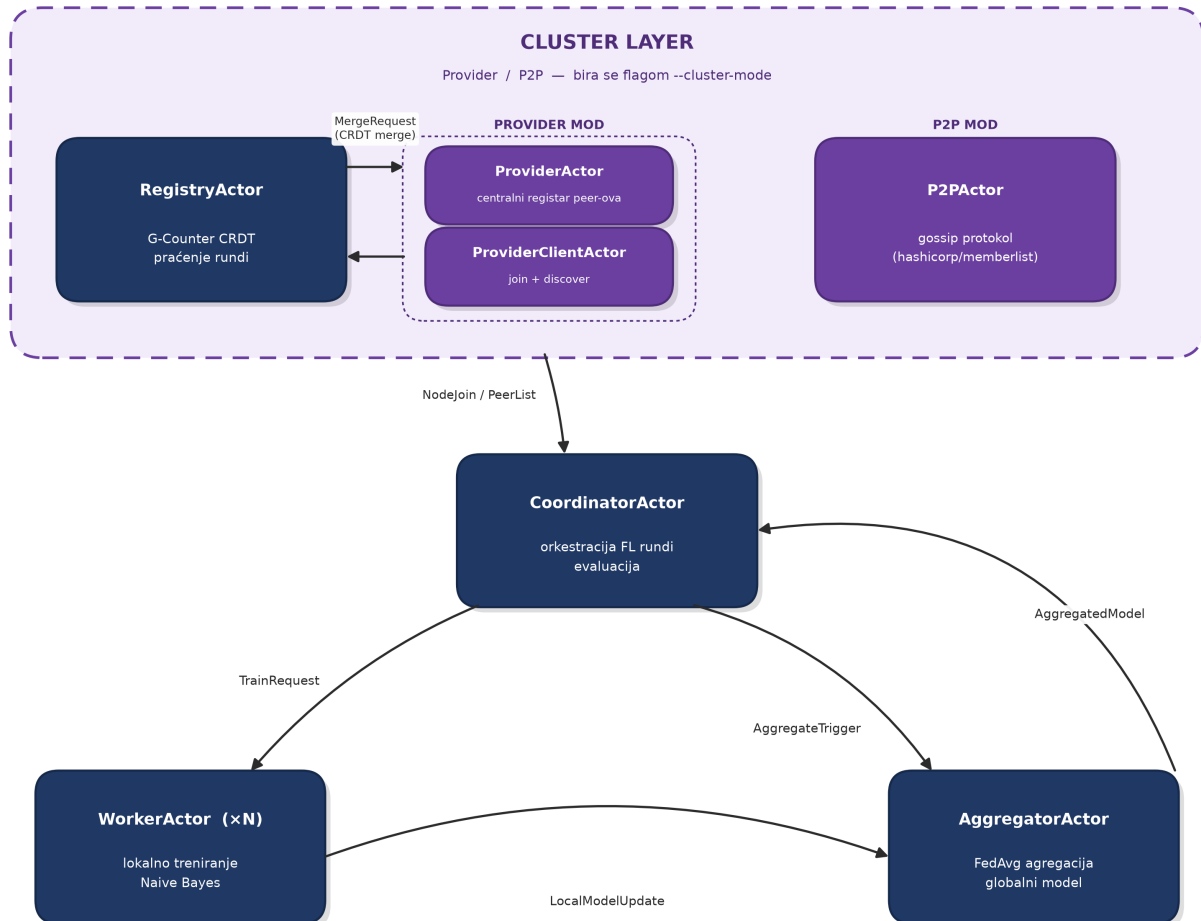
Test skup nikad ne napušta Coordinator čvor. Nakon svake FL runde Coordinator evaluira globalni model i ispisuje: Accuracy · Precision · Recall · F1-score. Rezultati se upoređuju sa centralnim modelom (baseline) treniranim na celom skupu. Cilj: federativni model postiže $\geq 95\%$ tačnosti centralnog modela.

Rezultat merenja (implementacija): federativni model postiže identičnu tačnost kao centralni baseline — accuracy 0.9865, odnos 1.0000 (cilj ≥ 0.95). Ovo je i matematički očekivano jer su NB parametri aditivni.

4. Aktori i komunikacija

► 4.1 Dijagram komunikacije (ažurirano)

Klaster sloj je konkretizovan kroz tri aktora umesto jednog generičkog ClusterActor-a: ProviderActor i ProviderClientActor za centralizovani (Provider) mod, i P2PActor za gossip (P2P) mod baziran na hashicorp/memberlist biblioteci. Mod se bira flagom `--cluster-mode` pri pokretanju.



► 4.2 Specifikacija aktora (ažurirano)

Aktor	Uloga	Prima poruke	Šalje poruke
CoordinatorActor	Orkestrira FL runde; inicira treniranje; vrši evaluaciju modela	StartTraining AggregatedModel	TrainRequest AggregateTrigger
WorkerActor (xN)	Lokalno trenira Naive Bayes na svojoj partitiji; ne deli sirove poruke	TrainRequest	LocalModelUpdate
AggregatorActor	Prikuplja lokalne modele; FedAvg sabiranje frekvencija reči; gradi globalni model	LocalModelUpdate AggregateTrigger	AggregatedModel
RegistryActor	Prati aktivne čvorove i završene FL runde koristeći G-Counter CRDT; replike se sinhronizuju preko mreže	NodeJoin, NodeLeave RoundComplete MergeRequest	RegistryState MergeRequest

Aktor	Uloga	Prima poruke	Šalje poruke
ProviderActor (novo)	Centralni registar u Provider modu: vodi listu aktivnih peer-ova i odgovara na upite	JoinCluster LeaveCluster DiscoverPeers	PeerList
ProviderClientActor (novo)	Redovan čvor u Provider modu: registruje se kod ProviderActor-a i uči listu peer-ova od njega	PeerList DiscoverNow (lokalno)	JoinCluster DiscoverPeers
P2PActor (novo)	Otkriva peer-ove gossip protokolom (hashicorp/memberlist); nema centralne tačke kvara	(memberlist event: NotifyJoin/Leave)	(gossip protokol, interno u memberlist-u)

► 4.3 Sadržaj ključnih poruka (ažurirano)

Sve poruke navedene ispod su implementirane kao Protocol Buffers poruke (vidi sekciju 6), tako da isti tip poruke putuje nepromenjen bilo da su pošiljalac i primalac u istom procesu ili na različitim mašinama.

Poruka	Sadržaj	Smer
TrainRequest	roundId: int64, datasetPartitionPath: string	Coordinator → Worker
LocalModelUpdate	roundId: int64, workerId: string, wordCounts: map<class,WordCounts>, classTotals: map<class,int64>	Worker → Aggregator
AggregateTrigger (precizirano)	roundId: int64, expectedWorkers: int64	Coordinator → Aggregator
AggregatedModel	roundId: int64, globalWordProbs: map<class,WordProbs>, classPriors: map<class,double>	Aggregator → Coordinator
MergeRequest	nodeId: string, counter: map<string,int64> — G-Counter state	Registry ↔ Registry
JoinCluster (novo)	self: PeerInfo {nodeId, addr}	ProviderClient → Provider
LeaveCluster (novo)	nodeId: string	ProviderClient → Provider
DiscoverPeers (novo)	replyTo: ReplyAddress {addr, actorId}	ProviderClient → Provider
PeerList (novo)	peers: repeated PeerInfo {nodeId, addr}	Provider → ProviderClient

5. Aktorski radni okvir (generički)

Radni okvir je generički — ne zavisi od federativnog učenja i upotrebljiv je za bilo koji aktorski sistem (paket pkg/actor). FL demo je demonstracija okvira, ne njegova definicija.

Element	Opis implementacije	Tip	Status
Actor interfejs	Go interfejs sa metodama Receive(ctx, msg) i ID() string	Obavezan	Implementirano
ActorSystem	Upravljanje životnim ciklusom: Spawn, Stop, Lookup aktora	Obavezan	Implementirano
Mailbox	Buffered Go channel — asinhrono slanje i primanje poruka	Obavezan	Implementirano
Behavior / Context	Context.Become(newBehavior) — promena ponašanja aktora u toku rada	Obavezan	Implementirano
Lifecycle eventi	OnStart, OnStop, OnError hooks — kreiranje, otkazivanje i recovery od panike	Obavezan	Implementirano
Udaljena komunikacija	gRPC + Protocol Buffers (google.protobuf.Any envelope); actor.Ref generalizuje lokalni PID i udaljenu referencu	Obavezan	Implementirano
Klasterovanje	P2P (gossip) + Provider cluster mod; bira se pri pokretanju	Dodatno x2	Implementirano (oba moda)

► 5.1 CRDT — G-Counter

RegistryActor koristi G-Counter CRDT za praćenje broja završenih rundi po čvoru, što garantuje eventual consistency bez centralne koordinacije:

- Struktura: map[nodeID]int — svaki čvor inkrementira isključivo svoju komponentu
- Increment: counter[localNodeID]++
- Merge: za svaki nodeID uzmi max(a[nodeID], b[nodeID])
- Value: sum(counter[nodeID]) za sve nodeID

Testirano: komutativnost, asocijativnost i idempotentnost merge operacije (jedinični testovi), i konvergencija dva registry replika preko stvarnog gRPC/TCP poziva između dva odvojena procesa.

► 5.2 Klasterovanje

Mod	Flag pri pokretanju	Mehanizam
P2P	--cluster-mode=p2p	Gossip protokol (hashicorp/memberlist) — čvorovi se otkrivaju međusobno, nema centralne tačke kvara
Provider	--cluster-mode=provider	Centralni registry čvor (ProviderActor) vodi listu aktivnih čvorova — jednostavnija konfiguracija

6. Detalji implementacije

Go 1.26+	Glavni programski jezik
gRPC + Protocol Buffers	Protokol za udaljenu komunikaciju aktora između čvorova — generic google.protobuf.Any envelope (pkg/actor/remote) plus tipizirane .proto šeme za FL, Registry i Cluster poruke
google.golang.org/grpc	Go gRPC biblioteka
github.com/hashicorp/memberlist	Gossip protokol za P2P klasterovanje
github.com/stretchr/testify	Unit i integracioni testovi (svi paketi)
Docker / docker-compose	Pokretanje sistema na više čvorova za demonstraciju

► 6.1 Struktura repozitorijuma (novo)

Putanja	Sadržaj
pkg/actor	Generički aktorski radni okvir (Actor, ActorSystem, Mailbox, Context/Become, lifecycle hooks)
pkg/actor/remote	gRPC transport: Envelope + google.protobuf.Any, Server, Ref
internal/dataset	Učitavanje, tokenizacija, train/test split, particionisanje SMS Spam Collection dataset-a
internal/nb	Naive Bayes: lokalno brojanje, FedAvg agregacija, predikcija, evaluacija
internal/fl	WorkerActor, AggregatorActor, CoordinatorActor + protobuf FL poruke (flpb)
internal/crdt	G-Counter CRDT
internal/registry	RegistryActor + protobuf registry poruke (registrypb)
internal/cluster	ProviderActor, ProviderClientActor, P2PActor + protobuf cluster poruke (clusterpb)
cmd/fl-demo	Jednoprocesni lokalni demo (FL runda + baseline poređenje)
cmd/cluster-node	Distribuirani binarni fajl: --role=coordinator worker aggregator, pokreće se kao zaseban proces/kontejner
Dockerfile, docker-compose.yml	Multi-stage build + 5 servisa (coordinator, aggregator, 3×worker) sa deljenim volume-om za particije

► 6.2 Validacija (novo)

Sledeće je stvarno pokrenuto i potvrđeno, ne samo napisano:

- Svi jedinični i integracioni testovi prolaze (go test ./...) za framework, dataset, NB, FL, CRDT, registry, cluster i remote transport pakete
- Federativna runda preko 5 realnih OS procesa (coordinator, aggregator, 3×worker) preko gRPC-a — accuracy 0.9865, identično centralnom baseline-u
- Ista runda preko 5 pravih Docker kontejnera (docker compose up) sa deljenim volume-om — identičan rezultat
- Provider mod: dva klijentska čvora otkrivaju jedan drugog isključivo preko centralnog ProviderActor-a, preko stvarnog TCP/gRPC poziva
- P2P mod: dva čvora se otkrivaju gossip protokolom (memberlist) preko stvarnih UDP/TCP soket-a, bez ijedne centralne tačke
- CRDT konvergencija dva registry replika (MergeRequest) preko stvarne mreže, nezavisno od redosleda gossip-a